



Exercícios de AED

Exercícios de Treino para o Exame Teórico de AED

124921 Nelson Rocha Ramos

Algoritmos e Estruturas de Dados (AED)
Departamento de Eletrónica, Telecomunicações e Informática
Ano Letivo 2025/2026

Nota: Estas perguntas foram retiradas de recursos disponíveis em vários Githubs da LECI (pex, luisbfsousa/leci e Paulo-Lacerda1/LECI); outras, baseadas nas disponíveis e criadas por mim. Não sou responsável por eventuais avarias que advenham da utilização deste recurso de treino/estudo.

Parte A (EM, VF)

Escolha Múltipla:

Resposta correta: 0.3 valores;
Resposta errada: -0.1 valores.

Verdadeiro / Falso:

Resposta correta: 0.2 valores;
Resposta errada: -0.1 valores.

Assinale **inequivocamente** uma alínea por questão.
Analise com cuidado.

Escolha Múltipla

- Sejam dadas duas funções $t(n)$ e $g(n)$, uma constante positiva c e um inteiro não negativo n_0 :
 - se $t(n) \geq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$.
 - se $t(n) \leq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $\Omega(g(n))$.
 - Ambas estão corretas.
 - Nenhuma está correta.
- Considere muitas instâncias distintas de um mesmo problema, todas da mesma dimensão e que foram resolvidas usando o mesmo algoritmo.
 - Se o esforço computacional para resolver cada instância for idêntico, a análise do algoritmo implicará resultados distintos para o pior caso e para o caso médio.
 - Se o esforço computacional depender da configuração de cada instância, a análise do algoritmo implicará resultados distintos para o pior caso e para o caso médio.
 - Ambas estão corretas.
 - Nenhuma está correta.
- Esta tabela representa o numero de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	4	8	16	32	64	128	256	512	1024
$M(n)$	3	4	5	6	7	8	9	10	11	12	13

Trata-se de um algoritmo com ordem de complexidade...?

- logarítmica.
 - linear.
 - quadrática.
 - Nenhuma está correta.
- Esta tabela representa o numero de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	4	8	16	32	64	128	256
$M(n)$	4	12	40	144	544	2112	8320	33024	131584

Trata-se de um algoritmo com ordem de complexidade...?

- logarítmica.
- linear.
- quadrática.
- Nenhuma está correta.

5. Esta tabela representa o número de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	3	4	5	6	7	8	9	10
$M(n)$	1	2	3	5	8	13	21	34	55	89

- (a) Da tabela, obtém-se a seguinte relação de recorrência: $M(n) = M(n - 1) + M(n - 2)$, para $n > 2$.
- (b) Trata-se de um algoritmo com ordem de complexidade exponencial.
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.
6. Considere o seguinte *array* de 6 elementos que se pretende ordenar usando o algoritmo "BubbleSort".

[6, 5, 4, 3, 2, 1]

- (a) São efetuadas **21 comparações** entre os elementos do *array*, para que seja ordenado por ordem crescente.
- (b) São efetuadas **5 trocas** entre elementos do *array*, para que seja ordenado por ordem crescente.
- (c) São efetuadas **15 trocas** entre elementos do *array*, para que seja ordenado por ordem crescente.
- (d) Nenhuma está correta.
7. Considere o *array* de 6 elementos que se pretende ordenar usando a versão do algoritmo "SelectionSort" em que se começa por colocar **o menor elemento na primeira posição**:

[6, 5, 4, 3, 2, 1]

- (a) São efetuadas **15 comparações** entre os elementos do *array*, para que seja ordenado por ordem crescente.
- (b) São efetuadas **3 trocas** entre elementos do *array*, para que seja ordenado por ordem crescente.
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.
8. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de **modo aleatório**. A ordem de complexidade dessa tarefa depende do algoritmo escolhido e será:

- (a) $O(n^2)$ se for usado o algoritmo BubbleSort.
- (b) $O(n \log n)$ se for usado o algoritmo HeapSort.
- (c) $O(n^2)$ se for usado o algoritmo de ordenação por seleção (SelectionSort).
- (d) Todas estão corretas.

9. Pretende-se resolver o problema das Torres de Hanoi para n discos:
- (a) Para $n = 2$ são necessários 4 movimentos de discos.
 - (b) Para $n = 3$ são necessários 8 movimentos de discos.
 - (c) O número de movimentos de discos efetuados é da ordem $O(2^n)$.
 - (d) Todas estão corretas.
10. Seja dada uma escada com n degraus, que podem ser subidos um a um, ou dois a dois, ou três a três, ou numa qualquer combinação dos movimentos anteriores (por exemplo, numa escada com três degraus pode subir-se um só degrau e depois dois de uma só vez).
- (a) Para $n = 3$, é possível subir a escada apenas de 4 maneiras diferentes.
 - (b) Para $n = 4$, é possível subir a escada apenas de 6 maneiras diferentes.
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
11. Seja dada uma árvore binária **total**, i.e., em que todos os níveis da árvore estão completamente preenchidos, com n nós:
- (a) O número de níveis é dado por $\lfloor \log_2 n \rfloor$
 - (b) O número de nós que são folhas é dado por $n \text{ div } 2$, em que div é o operador que determina o quociente da divisão inteira
 - (c) Ambas estão corretas
 - (d) Nenhuma está correta
12. Seja dada um árvore binária de altura **equilibrada** que armazena, de modo **ordenado**, n números inteiros.
- (a) No pior caso, determinar o valor do menor elemento armazenado na árvore é uma operação de complexidade $O(\log n)$.
 - (b) No pior caso, concluir que um dado número não pertence à árvore é uma operação de complexidade $O(\log n)$.
 - (c) Ambas estão corretas
 - (d) Nenhuma está correta
13. O *array* seguinte armazena, por níveis, os elementos de uma **árvore ternária**:
- $$[10, 6, 9, 2, 5, 7, 8, 0, 1, 3, -, -, -, -, -]$$
- (a) A árvore tem 7 folhas
 - (b) O elemento de valor 0 é filho do elemento de valor 9
 - (c) Ambas estão corretas
 - (d) Nenhuma está correta

14. O *array* seguinte armazena, por níveis, os elementos de uma Árvore Binária de Procura ("Binary Search Tree"). Pela ordem associada aos elementos da árvore:
- [8, 4, 12, 2, 6, 10, 14]
- (a) para procurar na árvore qualquer um dos elementos do conjunto $\{1, 3, 5, 7, 9, 11, 13, 15\}$, são sempre consultados 3 elementos, concluindo-se depois que o valor procurado não pertence à árvore.
 - (b) se forem sucessivamente procurados cada um dos elementos do conjunto $\{2, 4, 6, 8, 10, 12, 14\}$ é efetuado um total de 17 consultas a elementos da árvore.
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
15. Considere um grafo orientado $G(V, E)$ representado usando a lista ordenada dos seus nós e, para cada nó, a sua lista ordenada de adjacências:
- (a) No pior caso, verificar se um nó é isolado é uma op. de complexidade $O(E)$.
 - (b) No pior caso, adicionar uma aresta é uma operação de complexidade $O(V)$.
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
16. A teoria da complexidade computacional classifica diferentes problemas em classes de complexidade:
- (a) A **classe P** contém todos os problemas de decisão que podem ser resolvidos, em tempo polinomial, por um algoritmo determinista.
 - (b) A **classe NP** contém todos os problemas de decisão cuja solução **não** pode ser verificada, em tempo polinomial, por um algoritmo determinista.
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
17. Em C++, a **keyword** `auto` permite:
- (a) Deduzir, de modo automático, o tipo de uma variável em tempo de compilação.
 - (b) Inicializar, de modo automático, uma variável com o seu *default* value.
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
18. Qual das seguintes estruturas de dados (C++ *containers*) fornece operações eficientes — isto é, com complexidade $O(1)$ — tanto para a inserção como para a remoção de elementos nas duas extremidades?
- (a) `std::vector`.
 - (b) `std::deque`.
 - (c) `std::queue`.
 - (d) Nenhuma está correta.

19. Qual é a característica principal da estrutura de dados (C++ *container*) `std::set`?
- (a) Permite armazenar elementos duplicados.
 - (b) Permite o acesso aleatório aos elementos armazenados.
 - (c) Mantém os seus elementos ordenados.
 - (d) Nenhuma está correta.
20. Em C++, o algoritmo `std::count_if` permite:
- (a) Contar o número de ocorrências de um elemento específico (*target*) de uma estrutura de dados (*container*).
 - (b) Contar quantos elementos de uma estrutura de dados (*container*) satisfazem uma dada condição.
 - (c) Contar os elementos únicos de uma estrutura de dados (*container*).
 - (d) Nenhuma está correta.
-

Exame EN de 2024/2025

Mais algumas que encontrei do exame EN de 2024/2025... São poucas. Se encontrar mais, **avise-me**.

1. Sejam dadas duas funções $t(n)$ e $g(n)$, uma constante positiva c e um inteiro não negativo n_0 :
- (a) se $t(n) < c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$, i.e., $t(n)$ pertence a $O(g(n))$
 - (b) se $t(n) < c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $\Omega(g(n))$, i.e., $t(n)$ pertence a $\Omega(g(n))$
 - (c) se $t(n) \geq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$, i.e., $t(n)$ pertence a $O(g(n))$
 - (d) Nenhuma está correta
2. Considere muitas instâncias distintas de um mesmo problema, todas da mesma dimensão e que foram resolvidas usando o mesmo algoritmo.
- (a) Se o esforço computacional para resolver cada instância **não** foi idêntico, fará sentido analisar a eficiência computacional do algoritmo para o pior caso e o caso médio.
 - (b) Se o esforço computacional para resolver cada uma dessas instâncias **não** dependeu da configuração de cada instância, não faz qualquer sentido falar em pior caso e em caso médio.
 - (c) Ambas estão corretas
 - (d) Nenhuma está correta

3. A tabela seguinte representa o número de operações básicas efetuadas para sucessivos valores de n :

n	1	2	4	8	16	32	64	128	256	512	1024
$M(n)$	0	1	3	5	7	9	11	13	15	17	19

Trata-se de um algoritmo com ordem de complexidade:

- (a) logarítmica
 - (b) linear
 - (c) quadrática
 - (d) Nenhuma está correta
4. A tabela seguinte representa o número de operações básicas efetuadas para sucessivos valores de n :

n	1	2	4	8	16	32	64	128	256
$M(n)$	3	7	21	73	273	1057	4161	16513	65793

Trata-se de um algoritmo com ordem de complexidade:

- (a) logarítmica
 - (b) linear
 - (c) quadrática
 - (d) Nenhuma está correta
5. Quando se usa um algoritmo de procura exaustiva (*exhaustive search*), é geralmente necessário enumerar todas as possíveis soluções de um conjunto de n elementos; um passo essencial para a resolução do problema envolve a geração de todas as permutações desses elementos. Um exemplo de um problema onde isto é aplicado é (**Nota: enunciado estava muito desfocado, portanto não é exatamente assim, mas consegui +/- decifrar a questão**):

- (a) Problema da Mochila (*The Knapsack Problem*).
 - (b) Problema do Caixeiro Viajante (*The Traveling Salesman Problem*).
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.
6. Em C++, o algoritmo `std::count` permite:
- (a) Contar o número de ocorrências de um elemento específico (*target*) de uma estrutura de dados (*container*)
 - (b) Contar quantos elementos de uma estrutura de dados (*container*) satisfazem uma dada condição
 - (c) Contar os elementos únicos de uma estrutura de dados (*container*)
 - (d) Nenhuma está correta

Verdadeiro / Falso

- F 1. Um algoritmo é definido por uma sequência de instruções possivelmente ambíguas, que permite resolver instâncias de um problema num espaço de tempo finito.
- ✓ 2. Uma função $t(n)$ pertence a $\Theta(g(n))$ se existirem constantes $c_1, c_2 > 0$ e n_0 tal que $c_1g(n) \leq t(n) \leq c_2g(n)$, para todo o $n > n_0$.
- ✓ 3. Ao efetuar, usando o algoritmo clássico, o produto de matrizes $A \times B = C$, em que a matriz A tem 10 linhas e 20 colunas e a matriz B tem 20 linhas e 10 colunas, são efetuadas **2000** multiplicações.
- F 4. Um algoritmo que gera todas as permutações de um conjunto de n elementos é um algoritmo de ordem de complexidade exponencial, i.e., pertence a $O(2^n)$.
- ✓ 5. No caso médio, a versão **iterativa** do algoritmo de Pesquisa Binária num *array* ordenado tem complexidade $O(\log n)$.
- F 6. Quando se usa a técnica da programação dinâmica, é necessário resolver repetidas vezes os mesmos sub-problemas.
- F 7. $0 + 3 + 6 + \dots + 3(n-1) + 3n = 3(n^2 - n)/n$.
- ✓ 8. $1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1$.
- F 9. Quando se ordena um dado *array* de n elementos usando o algoritmo *HeapSort*, o primeiro passo é, habitualmente, transformar o *array* dado numa **MIN-HEAP**.
- ✓ 10. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de **modo aleatório**. Se for usado o algoritmo *HeapSort*, a ordem de complexidade dessa tarefa será $O(n \log n)$.
- ✓ 11. Numa árvore binária, o número máximo de nós do nível i é 2^i , considerando que a raiz da árvore pertence ao nível zero ($i=0$).
- F 12. Na travessia **pós-ordem** de uma árvore binária, todos os elementos da subárvore direita da raiz são visitados antes dos elementos da subárvore esquerda da raiz.
- ✓ 13. Numa árvore binária de pesquisa ("Binary Search Tree"), a subárvore esquerda de um dado nó **não** pode conter elemento de valor superior a esse mesmo nó.
- F 14. Uma árvore AVL é uma árvore binária equilibrada em altura em que, para cada nó, as alturas das suas duas subárvores diferem, **sempre**, de uma unidade.
- ✓ 15. Um grafo (não orientado) completo com n vértices tem $(n^2 - n)/2$ arestas.
- ✓ 16. Se um grafo orientado é **fortemente conexo**, não tem qualquer vértice isolado.
- ✓ 17. Dado um grafo, um circuito Hamiltoniano é um caminho que, partindo de um qualquer nó, atravessa, uma única vez, cada um dos outros vértices do grafo e regressa ao inicial.
- ✓ 18. Considere um grafo orientado $G(V, E)$ representado usando uma lista de vértices e, para cada vértice, a sua lista de adjacências. Se o grafo for completo, existem V listas de adjacências e o número total de nós definindo as várias listas de adjacências é $V \times (V - 1)$.

- F 19. A travessia por níveis ("Breadth-First Traversal") de um grafo orientado é habitualmente realizada usando uma pilha ("Stack").
- V 20. Instâncias de grande dimensão do Problema do Caixeiro Viajante ("The Traveling Salesperson Problem") **não** são habitualmente resolúveis em tempo útil.
-

Exame EN de 2024/2025

- F 1. Um algoritmo é definido por uma sequência de instruções **não-ambíguas** que permite resolver instâncias de um problema num espaço de tempo **indefinido**.
- V 2. Uma função $t(n)$ será da ordem $\Omega(g(n))$, i.e. $t(n)$ pertence a $\Omega(g(n))$, se existir uma constante positiva c e um inteiro não-negativo n_0 tais que $cg(n) \leq t(n)$, para todo o $n > n_0$.
- V 3. Seja $t(n) = 1000n^2 + n^3$. Então, $t(n)$ é da ordem de $O(n^3)$, i.e., $t(n)$ pertence a $O(n^3)$.
- F 4. Ao efetuar, usando o algoritmo clássico, o produto de matrizes $(A \times B) \times C = D$, em que a matriz A tem 2 linhas e 3 colunas, a matriz B tem 3 linhas e 4 colunas e a matriz C tem 4 linhas e 5 colunas, são efetuadas, **no total, 120** multiplicações.
- V 5. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de **modo aleatório**. Se for usado o algoritmo **Selection Sort**, e consideradas o número de comparações efetuadas, a ordem de complexidade dessa tarefa será $O(n^2)$.
- V 6. Um algoritmo que gera todos os **subconjuntos** de um conjunto de n elementos é um algoritmo de ordem de complexidade exponencial, i.e., pertence a $O(2^n)$.
7. A técnica de Programação Dinâmica pode ser utilizada para desenvolver um algoritmo para determinar as combinações $C(n, p)$ – i.e., os coeficientes binomiais.
- F 8. A combinação das técnicas de Programação Dinâmica e "memoization" é útil, mas continuam a ser executados cálculos **redundantes**.
- V 9. $0 + 10 + 20 + 30 + \dots + 10(n-1) + 10n = 5(n^2 + n)$.
- V 10. $0 + 1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1$.

Não encontrei mais V/F do Exame EN de 2024/2025... Se encontrar, **avise-me**.

Parte B

Nota 1: As funções desenvolvidas deverão ser apresentadas usando a linguagem C e não devem ser utilizadas variáveis globais.

Nota 2: Todo o código deverá, em preparação para o Exame Teórico, ser todo realizado à mão, em papel. No exame, não teremos acesso a IDEs e o tempo disponível será diminuto, no qual teremos três questões deste gênero, após a Parte A (escolha múltipla, V/F).

Nota 3: São apresentadas as questões da EN 23/24, EN 24/25 e ER 24/25. Do último exame, estão em falta uma questão de cada, que não consegui encontrar... São apresentadas também algumas questões criadas por mim.

Exame AED 2023-2024 EN

Q1

Uma possível estratégia para efetuar a ordenação de um vetor por **Seleção Linear** consiste basicamente, para cada passo, em procurar a primeira ocorrência do menor elemento de um (sub-)vetor e trocá-lo depois com o primeiro. Considere, sem perda de generalidade, vetores cujos elementos são números inteiros.

a) (2.0 valores)

Construa uma função iterativa que ordene um vetor usando a estratégia indicada. A função deverá ter o seguinte protótipo:

```
void SelectionSort(int* a, unsigned int n);
```

b) (1.0 valores)

Análise o número de comparações — associadas a elementos do vetor — efetuadas pelo algoritmo desenvolvido. Escreva:

- uma expressão inicial usando somatórios;
- a expressão final para o número total de comparações;
- a ordem de complexidade.

c) (1.0 valores)

Análise o número de trocas — associadas a elementos do vetor — efetuadas pelo algoritmo. Estude o comportamento no:

- Melhor Caso
- Pior Caso
- Caso Médio

Indique configurações do vetor que conduzam a cada um destes casos.

$$\sum_{k=1}^n \frac{1}{k} \approx \ln(n) + \gamma, \quad \gamma = 0,577\dots$$

Q2

Considere uma estrutura de dados que permite representar uma sequência de números naturais usando uma **lista ligada**, em que os sucessivos elementos se encontram ordenados de modo não-decrescente, podendo existir elementos repetidos.

O acesso à lista é feito a partir de um **nó cabeçalho**, que contém:

- um ponteiro para o primeiro nó da lista;
- um valor inteiro que indica o número de elementos da sequência.

a) (2.0 valores)

Dadas duas sequências — possivelmente vazias — representadas usando essa estrutura, desenvolva uma função iterativa que devolva a nova sequência resultante da sua fusão ordenada. A função deverá ter o seguinte protótipo:

```
header_pointer Merge(header_pointer seq_1, header_pointer seq_2);
```

b) (1.0 valores)

Considere que as sequências têm tamanhos n_1 e n_2 , com $n_1 \geq 0$ e $n_2 \geq 0$. Faça a análise da complexidade do algoritmo relativamente ao número de comparações entre elementos:

- Melhor Caso
- Pior Caso

Q3

Considere o TAD **Árvore Binária de Inteiros**, em que cada nó armazena um número inteiro. Considere que os elementos da árvore se encontram ordenados (*in-order*) e que não existem elementos repetidos.

a) (1.0 valores)

Desenvolva uma função recursiva que, dado o ponteiro para o nó raiz de uma árvore, conte o número de nós que **não são folhas**. A função deverá ter o protótipo:

```
unsigned int CountNonLeafNodes(pointer root);
```

b) (0.5 valores)

Que tipo de travessia é efetuada pelo algoritmo da função anterior?

c) (1.5 valores)

Desenvolva uma função recursiva eficiente — que **não aceda a nós desnecessários** — que, dada a raiz da árvore, conte o número de elementos pertencentes ao intervalo

$$[a, b], \quad b \geq a.$$

A função deverá ter o seguinte protótipo:

```
unsigned int CountValuesInRange(pointer root, int a, int b);
```

Exame AED 2023-2024 ER

Não tenho nenhuma questão deste exame. Se tiver alguma, mande-ma e eu adiciono a este ficheiro.

Exame AED 2024-2025 EN

Q1

Dado um vetor ordenado, cujos elementos são números inteiros positivos, pretende-se criar um novo vetor contendo os mesmos elementos, mas agora organizados em duas partições:

1. contém os números ímpares, por ordem crescente;
2. contém os números pares, por ordem decrescente.

Por exemplo, o vetor

[1, 2, 3, 4, 5]

origina

[1, 3, 5, 4, 2].

Construa de forma iterativa a função que, dado um vetor com n elementos, devolve um novo vetor cujos elementos estão organizados como indicado acima. O vetor original deve permanecer inalterado.

A função tem o protótipo:

```
unsigned int* Partition(unsigned int* a, unsigned int n);
```

(3.0 valores).

Q2

Considere uma Lista Ligada Simples que representa uma sequência de números naturais, sem garantia de ordenação, e podendo existir elementos repetidos. O acesso é feito usando o ponteiro para o primeiro nó da lista.

Os nós da lista são definidos como:

```
struct node { unsigned int value; struct node* next; };  
typedef struct node* pointer;
```

a) (2.0 valores)

Desenvolva uma função iterativa que apague os elementos repetidos de uma dada lista, e devolva o número de elementos apagados.

Por exemplo, a sequência:

$$\{1, 5, 3, 5, 2, 2, 1, 4, 5\}$$

é transformada em:

$$\{1, 5, 3, 2, 4\}.$$

A função tem o seguinte protótipo:

```
unsigned int DeleteRepetitions(pointer p);
```

b) (0.5 valores)

Pretende-se analisar o esforço computacional da função desenvolvida para o **melhor caso**, contando o número total de visitas a nós da lista. Considere uma lista com 10 elementos:

- Que configuração da lista corresponde ao melhor caso?
- Qual é o número total de visitas a nós efetuadas pela função?

c) (1.0 valores)

Faça a mesma análise anterior, agora para o **pior caso**. Indique:

- a configuração da lista que conduz ao pior caso;
- o número total de visitas a nós efetuadas pela função.

Q3

Falta-me esta questão para este exame (EN 2024-2025). Não consegui encontrá-la... Caso consiga, por favor avise-me.

Exame AED 2024-2025 ER

Q1

Falta-me esta questão para este exame (ER 2024-2025). Não consegui encontrá-la... Caso consiga, por favor avise-me.

Q2

Considere uma **lista ligada** que permite representar uma sequência de números naturais, cujos elementos **não se encontram ordenados**.

Pretende-se criar uma nova lista, com o mesmo número de nós, e alocando memória para os seus nós, organizada do seguinte modo:

- os números pares encontram-se no início da nova lista, pela **ordem inversa** que têm na lista original,
- os números ímpares encontram-se no final da nova lista, pela **ordem que têm na lista original**.

Por exemplo, a sequência:

$\{1, 5, 3, 2, 4, 7, 9, 8\}$

é transformada na sequência:

$\{8, 4, 2, 1, 5, 3, 7, 9\}$.

O acesso à lista é feito usando o ponteiro para o primeiro nó da lista.

Os nós da lista são definidos como:

```
struct node { unsigned int value; struct node* next; };  
typedef struct node* pointer;
```

A função tem o seguinte protótipo e devolve um ponteiro para o primeiro nó da lista criada, ou NULL se a lista original se encontrar vazia:

`pointer Partition(pointer p);`

A lista original deve permanecer inalterada. **Não deve usar arrays auxiliares.**

(2.0 valores).

Q3

Considere uma **Árvore Binária de Procura (ABP)**, com os seus elementos ordenados e armazenando números inteiros positivos, sem repetições.

Os nós da árvore são definidos como:

```
struct node { unsigned int value; struct node* left; struct node* right; };  
typedef struct node* pointer;
```

a) (1.5 valores)

Desenvolva uma **função iterativa** que, dado o ponteiro para o nó raiz de uma árvore, procura um dado valor inteiro e devolve 0 (falso) ou 1 (verdadeiro), consoante o resultado.

O protótipo da função é:

```
unsigned int Search(pointer root, unsigned int number);
```

b) (2 valores)

Desenvolva uma **função recursiva** que, dado o ponteiro para o nó raiz de uma primeira árvore (**root1**) e o ponteiro para o nó raiz de uma segunda árvore (**root2**), conta quantos elementos da primeira árvore pertencem à segunda.

Use obrigatoriamente a função da alínea anterior. Não deve usar variáveis globais.

O protótipo da função é:

```
unsigned int CountContainedIn(pointer root1, pointer root2);
```

Exercícios-Tipo feitos por mim

Vetores

Q1

Dado um vetor ordenado crescentemente de números inteiros positivos, pretende-se criar um novo vetor que contenha os mesmos elementos, mas organizados agora em três partições distintas, segundo o critério:

1. A primeira partição contém todos os múltiplos de 5, por ordem crescente.
2. A segunda partição contém todos os números pares que não são múltiplos de 5, por ordem decrescente.
3. A terceira partição contém todos os restantes elementos (ímpar não múltiplo de 5), também por ordem decrescente.

Por exemplo, o vetor

[1, 2, 3, 4, 5, 6, 8, 10, 11]

deve originar o vetor:

[5, 10, 8, 6, 4, 2, 11, 3, 1]

O vetor original não deve ser alterado. A função a implementar tem o protótipo:

```
unsigned int* PartitionThree(unsigned int* a, unsigned int n);
```

a)

Implemente a função iterativa `PartitionThree`, que devolve o novo vetor com as três partições.

b)

Considere que o vetor tem tamanho n . Analise o número de comparações efetuadas pelo algoritmo. Inclua:

- Uma expressão inicial com somatórios.
- A dedução da expressão final.
- A ordem de complexidade temporal.

c)

Assuma agora que pretende minimizar o número de escritas (*writes*) no vetor resultante. Analise o comportamento do algoritmo no:

- Melhor Caso
- Pior Caso
- Caso Médio

Indique configurações do vetor original que conduzam a cada um destes casos.

Q2

Considere um vetor de números inteiros positivos, não necessariamente ordenado. Pretende-se criar um novo vetor contendo:

1. todos os números **primos**, mantendo a sua ordem relativa original;
2. seguidos de todos os números **não primos**, por ordem crescente.

Por exemplo, o vetor:

[8, 5, 9, 2, 4, 7, 1]

deve originar o vetor:

[5, 2, 7, 1, 4, 8, 9].

O vetor original não deve ser alterado.

a)

Implemente uma função iterativa com o protótipo:

```
unsigned int* PrimeStablePartition(unsigned int* a, unsigned int n);
```

b)

Analise o número de comparações efetuadas, incluindo:

- expressão inicial com somatórios;
- dedução da forma fechada;
- ordem de complexidade temporal.

c)

Considere o número de escritas no vetor final. Analise o Melhor, Pior e Caso Médio e indique configurações do vetor que conduzam a cada caso.

Q3

Considere dois vetores de inteiros positivos, ambos ordenados crescentemente, com tamanhos n_1 e n_2 . Pretende-se construir um novo vetor que resulta de intercalar (interleave) os elementos de ambos, obedecendo às regras:

1. a cada passo deve ser escolhido o menor elemento disponível;
2. em caso de empate, deve ser escolhido o elemento do **primeiro vetor**.

Por exemplo:

$$v_1 = [1, 3, 6, 9], \quad v_2 = [2, 3, 5, 7]$$

produz o vetor:

$$[1, 2, 3, 3, 5, 6, 7, 9].$$

a)

Implemente a função iterativa:

```
unsigned int* InterleavedMerge(unsigned int* v1,  
                               unsigned int n1, unsigned int* v2, unsigned int n2);
```

b)

Analise o número de comparações realizadas no Melhor Caso e no Pior Caso, justificando as configurações de cada um.

Q4

Dado um vetor **ordenado** crescentemente de N números inteiros positivos, desenvolva uma função que remova todas as ocorrências repetidas de elementos, compactando o vetor. O resultado da função deverá ser o novo tamanho do vetor após a remoção. O vetor deve ser modificado no local (in-place).

a)

Construa a função iterativa que implemente a compressão indicada. O protótipo é:

```
unsigned int CompressArray(unsigned int* a, unsigned int n);
```

Exemplo:

$$[1, 1, 2, 3, 3, 3, 4] \longrightarrow [1, 2, 3, 4, \dots]$$

A função deve devolver 4.

b)

Assuma que o vetor tem N elementos. Analise o **número total de comparações** entre elementos efetuadas pelo algoritmo. Escreva:

- a expressão final,
- a ordem de complexidade temporal.

c)

Assuma que o vetor tem N elementos. Analise o **número total de escritas** (atribuições) efetuadas no vetor. Indique:

- a configuração do vetor que conduz ao **Pior Caso** (máximo de escritas);
- o número total de escritas nesse caso.

Exercícios-Tipo feitos por mim

Listas Ligadas

Q1

Considere uma Lista Ligada Simples onde os nós contêm números inteiros. O acesso à lista é feito apenas através do ponteiro para o primeiro nó (`pointer head`). Os nós são definidos da seguinte forma:

```
typedef struct node { int value; struct node* next; } Node;
typedef Node* pointer;
```

Pretende-se reorganizar os elementos desta lista de modo que todos os números negativos apareçam antes dos números não negativos (zero ou positivos). A **ordem relativa** dos elementos dentro de cada partição não deve ser alterada.

Por exemplo, a lista

$$\{1, -5, 3, -2, 0, 4\}$$

deve resultar em

$$\{-5, -2, 1, 3, 0, 4\}.$$

a)

Desenvolva uma função iterativa que rearranje a lista ligada conforme a estratégia indicada, movendo os nós existentes e alterando apenas os ponteiros `next`. A função deve ter o seguinte protótipo:

```
pointer PartitionList(pointer head);
```

b)

Considere que a lista tem N elementos. Faça a análise da complexidade do algoritmo implementado na parte (a) em termos do número total de vezes que os nós da lista são visitados (comparações). Escreva a expressão final para o número de visitas realizadas. Indique a ordem de complexidade do algoritmo.

c)

Análise o número de operações de re-ligação de ponteiros (`next` modificados) efetuadas pelo algoritmo. Analise o comportamento no:

- Melhor Caso
- Pior Caso

Indique a configuração da lista que conduz a esses casos e o número de re-ligações em cada um.

Q2

Considere uma Lista Ligada Simples cujos nós armazenam inteiros, com a mesma estrutura da LL do exercício anterior. Pretende-se remover todos os nós situados em **posições pares** (2^{a} , 4^{a} , 6^{a} , ...), assumindo que a primeira posição é a 1.

Por exemplo:

$$\{10, 20, 30, 40, 50\}$$

é transformada em:

$$\{10, 30, 50\}.$$

a)

Desenvolva uma função iterativa que elimina os nós das posições pares e devolve o número de nós removidos. Protótipo:

```
unsigned int RemoveEvenPositions(pointer head);
```

b)

Analise o número total de visitas a nós da lista no Melhor Caso e no Pior Caso, para uma lista com N elementos. Indique a configuração da lista que conduz a cada caso.

Q3

Considere uma Lista Ligada Simples que armazena inteiros (podem existir repetidos). Pretende-se gerar uma nova lista, **reutilizando os nós existentes**, contendo:

1. todos os números negativos, mantendo ordem relativa;
2. seguidos dos zeros, mantendo ordem relativa;
3. seguidos dos positivos, invertendo ordem relativa.

Não pode ser alocado novo espaço para nós.

a)

Implemente a função iterativa:

```
pointer ThreeWayStablePartition(pointer head);
```

b)

Analise o número de comparações (visitas a nós) efetuadas. Indique a expressão final e a ordem de complexidade.

c)

Analise o número de re-ligações de ponteiros (next modificados) no Melhor Caso e no Pior Caso.

Q4

Considere uma Lista Ligada Simples de N elementos. Desenvolva uma função que inverta a lista em **blocos de K nós**. Os nós devem ser movidos e os ponteiros alterados.

a)

Construa uma função iterativa que implemente a inversão em blocos. O protótipo é:

```
pointer ReverseInBlocks(pointer head, unsigned int K);
```

Exemplo:

$$\{1, 2, 3, 4, 5, 6\}, \quad K = 3 \quad \longrightarrow \quad \{3, 2, 1, 6, 5, 4\}$$

b)

Assuma que a lista tem N elementos e que N é múltiplo de K . Analise a complexidade em termos do **número total de visitas a nós**. Escreva:

- a expressão final,
- a ordem de complexidade.

c)

Calcule o número total de **atribuições de ponteiros** (`->next`) efetuadas pelo algoritmo, assumindo que N é múltiplo de K , indicando o seu raciocínio.

Exercícios-Tipo feitos por mim

Árvores Binárias

Q1

Considere uma Árvore Binária de Inteiros ordenada (in-order), sem elementos repetidos.

a)

Implemente uma função recursiva que devolva a soma dos valores de todas as folhas da árvore. Protótipo:

```
int SumLeafValues(pointer root);
```

b)

Indique qual o tipo de travessia que a função realiza.

c)

Analise a complexidade temporal no Melhor Caso e no Pior Caso, indicando configurações da árvore que conduzam a cada situação.

Q2

Considere uma Árvore Binária de Pesquisa (BST) sem valores repetidos.

a)

Implemente uma função recursiva eficiente — que não visite nós desnecessários — que conte o número de elementos **estritamente menores** do que um dado valor X . Protótipo:

```
unsigned int CountLessThan(pointer root, int X);
```

b)

Justifique porque é possível evitar explorar subárvores inteiras na sua solução.

c)

Analise o número máximo e mínimo de nós visitados, indicando configurações que conduzam ao Melhor e ao Pior Caso.

Q3

Considere uma Árvore Binária de Pesquisa (ABP) onde cada nó armazena um número inteiro. Considere que não existem elementos repetidos.

a)

Desenvolva uma função recursiva que receba o vetor correspondente a uma travessia **pré-ordem** (Pre-order) e o seu tamanho, e reconstrua a ABP original.

Protótipo:

```
pointer BuildBSTFromPreOrder(int* preOrderArray, unsigned int n);
```

b)

Determine o **Melhor Caso** para o número de comparações efetuadas na construção da árvore. Indique:

- a configuração da ABP original que conduz ao Melhor Caso;
- a ordem de complexidade nesse caso.

c)

Que tipo de travessia adicional seria necessária para reconstruir a árvore se esta **não fosse** uma ABP (apenas uma árvore binária genérica)? Assuma que é permitido utilizar mais uma travessia além do pré-ordem.

Q4

Considere uma **árvore binária de inteiros**, cujos nós são definidos como:

```
struct node {  
    int value;  
    struct node* left;  
    struct node* right;  
};  
typedef struct node* pointer;
```

a)

Desenvolva um trio de funções que calcule a soma dos elementos do **nível de maior profundidade** da árvore (ou seja, o nível das folhas mais distantes da raiz). A função deverá ser **recursiva**. O protótipo da principal é:

```
int SumMaxDepth(pointer root);
```

Sugestão: Crie outras duas funções recursivas `MaxDepth(pointer root)` e `SumLevel(pointer root, int level)`, e use-as na sua função recursiva.

b)

Assuma que a árvore tem N nós. Indique:

- a ordem de complexidade temporal,
- a ordem de complexidade espacial (memória auxiliar) da sua função.

c)

Se o requisito fosse calcular a soma dos elementos do **nível mais próximo da raiz** (os filhos diretos da raiz), indique qual seria a alteração mais eficiente na solução da alínea (a).

Parte A (EM, VF)

Respostas

Escolha Múltipla:

Resposta correta: 0.3 valores;
Resposta errada: -0.1 valores.

Verdadeiro / Falso:

Resposta correta: 0.2 valores;
Resposta errada: -0.1 valores.

Se detetar algum erro, avise-me, por favor, para eu corrigir!

Escolha Múltipla

1. Sejam dadas duas funções $t(n)$ e $g(n)$, uma constante positiva c e um inteiro não negativo n_0 :

- (a) se $t(n) \geq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$.
- (b) se $t(n) \leq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $\Omega(g(n))$.
- (c) Ambas estão corretas.
- (d) **Nenhuma está correta.**

Resolução: Por definição:

- $t(n) \in O(g(n))$ implica $t(n) \leq c g(n)$;
- $t(n) \in \Omega(g(n))$ implica $t(n) \geq c g(n)$.

As desigualdades apresentadas nas alíneas (a) e (b) estão trocadas. Logo, nenhuma está correta.

2. Considere muitas instâncias distintas de um mesmo problema, todas da mesma dimensão e que foram resolvidas usando o mesmo algoritmo.
- (a) Se o esforço computacional para resolver cada instância for idêntico, a análise do algoritmo implicará resultados distintos para o pior caso e para o caso médio.
 - (b) **Se o esforço computacional depender da configuração de cada instância, a análise do algoritmo implicará resultados distintos para o pior caso e para o caso médio.**
 - (c) Ambas estão corretas.
 - (d) Nenhuma está correta.

Resolução: A distinção entre pior caso e caso médio só faz sentido quando diferentes instâncias exigem esforços distintos. Se todas exigirem o mesmo esforço, esses conceitos coincidem.

3. Esta tabela representa o numero de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	4	8	16	32	64	128	256	512	1024
$M(n)$	3	4	5	6	7	8	9	10	11	12	13

Trata-se de um algoritmo com ordem de complexidade...?

- (a) **logarítmica.**
- (b) linear.
- (c) quadrática.
- (d) Nenhuma está correta.

Resolução: Sempre que n duplica, $M(n)$ aumenta aproximadamente de uma unidade, o que caracteriza crescimento $\Theta(\log n)$.

4. Esta tabela representa o numero de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	4	8	16	32	64	128	256
$M(n)$	4	12	40	144	544	2112	8320	33024	131584

Trata-se de um algoritmo com ordem de complexidade...?

- (a) logarítmica.
- (b) linear.
- (c) **quadrática.**
- (d) Nenhuma está correta.

Resolução: Se n duplica e o número de operações quadruplica, trata-se de crescimento proporcional a n^2 .

5. Esta tabela representa o número de operações básicas efetuadas por um determinado algoritmo para sucessivos valores de n .

n	1	2	3	4	5	6	7	8	9	10
$M(n)$	1	2	3	5	8	13	21	34	55	89

- (a) Da tabela, obtém-se a seguinte relação de recorrência: $M(n) = M(n - 1) + M(n - 2)$, para $n > 2$.
- (b) Trata-se de um algoritmo com ordem de complexidade exponencial.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: A sequência apresentada corresponde à sequência de Fibonacci, cuja complexidade cresce exponencialmente, da ordem de $O(\varphi^n)$.

6. Considere o seguinte *array* de 6 elementos que se pretende ordenar usando o algoritmo "BubbleSort".

[6, 5, 4, 3, 2, 1]

- (a) São efetuadas 21 comparações entre os elementos do *array*, para que seja ordenado por ordem crescente.
- (b) São efetuadas 5 trocas entre elementos do *array*, para que seja ordenado por ordem crescente.
- (c) **São efetuadas 15 trocas entre elementos do *array*, para que seja ordenado por ordem crescente.**
- (d) Nenhuma está correta

Resolução: No BubbleSort clássico, o número de trocas é, no pior caso (*array* invertido), igual ao número de comparações:

$$(6 - 1) + (6 - 2) + \dots + 1 = n(n - 1)/2 = 15 \text{ comparações} \Rightarrow 15 \text{ trocas.}$$

7. Considere o *array* de 6 elementos que se pretende ordenar usando a versão do algoritmo "SelectionSort" em que se começa por colocar o menor elemento na primeira posição:

$[6, 5, 4, 3, 2, 1]$

- (a) São efetuadas 15 comparações entre os elementos do *array*, para que seja ordenado por ordem crescente.
- (b) São efetuadas 3 trocas entre elementos do *array*, para que seja ordenado por ordem crescente.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: O SelectionSort executa sempre $\frac{n(n-1)}{2}$ comparações, independentemente da configuração inicial. Neste caso, o número de trocas **efetivas** é 3 (4 e 5 trocam com eles próprios. Se se considerarem essas trocas, o número de trocas é 5).

8. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de modo aleatório. A ordem de complexidade dessa tarefa depende do algoritmo escolhido e será:

- (a) $O(n^2)$ se for usado o algoritmo BubbleSort.
- (b) $O(n \log n)$ se for usado o algoritmo HeapSort.
- (c) $O(n^2)$ se for usado o algoritmo de ordenação por seleção (SelectionSort).
- (d) **Todas estão corretas.**

Resolução: As complexidades indicadas estão corretas.

9. Pretende-se resolver o problema das Torres de Hanoi para n discos:

- (a) Para $n = 2$ são necessários 4 movimentos de discos.
- (b) Para $n = 3$ são necessários 8 movimentos de discos.
- (c) **O número de movimentos de discos efetuados é da ordem $O(2^n)$.**
- (d) Todas estão corretas.

Resolução: 3 e 7 $\neq$ 4 e 8; o número exato de movimentos é $2^n - 1$.

10. Seja dada uma escada com n degraus, que podem ser subidos um a um, ou dois a dois, ou três a três, ou numa qualquer combinação dos movimentos anteriores (por exemplo, numa escada com três degraus pode subir-se um só degrau e depois dois de uma só vez).

- (a) **Para $n = 3$, é possível subir a escada apenas de 4 maneiras diferentes.**
- (b) Para $n = 4$, é possível subir a escada apenas de 6 maneiras diferentes.
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.

Resolução: A recorrência é $T(n) = T(n-1) + T(n-2) + T(n-3)$, $T(0) = 1$, $T(1) = 1$, $T(2) = 2$. Para $n = 4$ existem 7 soluções, não 6.

11. Numa árvore binária total, i.e., em que todos os níveis da árvore estão completamente preenchidos, com n nós:

- (a) O número de níveis é dado por $\lfloor \log_2 n \rfloor$.
- (b) O número de nós que são folhas é dado por $n \text{ div } 2$, em que div é o operador que determina o quociente da divisão inteira.
- (c) Ambas estão corretas.
- (d) **Nenhuma está correta.**

Resolução: Numa árvore binária total, se a altura é h (raiz no nível 0), então: $n = 2^{h+1} - 1 \Rightarrow h = \log_2(n+1) - 1$. Logo, o número de níveis é $h+1 = \log_2(n+1)$, não $\lfloor \log_2 n \rfloor$. Além disso, o número de folhas é $2^h = \frac{n+1}{2}$, e não $n \text{ div } 2$.

12. Seja dada um árvore binária total, i.e., em que todos os níveis da árvore estão completamente preenchidos, com n nós.

- (a) No pior caso, determinar o valor do menor elemento armazenado na árvore é uma operação de complexidade $O(\log n)$.
- (b) No pior caso, concluir que um dado número não pertence à árvore é uma operação de complexidade $O(\log n)$.
- (c) Ambas estão corretas.
- (d) **Nenhuma está correta.**

Resolução: Uma ABT (total) **não** é uma ABP (procura). No pior caso, encontrar o menor elemento / verificar a inexistência de um valor arbitrário, é $O(n)$ (necessário percorrer todos os nós). Essas afirmações aplicam-se a ABPs.

13. O *array* seguinte armazena, por níveis, os elementos de uma árvore ternária:

$[10, 6, 9, 2, 5, 7, 8, 0, 1, 3, -, -, -, -, -]$

- (a) A árvore tem 7 folhas.
- (b) O elemento de valor 0 é filho do elemento de valor 9.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: Numa árvore ternária representada por níveis, cada nó tem até 3 filhos. Observando a estrutura, existem exatamente 7 folhas e o nó com valor 0 é efetivamente filho do nó 9.

Sugiro que desenhe neste espaço a árvore ternária... será + fácil perceber.

14. O *array* seguinte armazena, por níveis, os elementos de uma Árvore Binária de Procura ("Binary Search Tree"). Pela ordem associada aos elementos da árvore:

[8, 4, 12, 2, 6, 10, 14]

- (a) para procurar na árvore qualquer um dos elementos do conjunto $\{1, 3, 5, 7, 9, 11, 13, 15\}$, são sempre consultados 3 elementos, concluindo-se depois que o valor procurado não pertence à árvore.
- (b) se forem sucessivamente procurados cada um dos elementos do conjunto $\{2, 4, 6, 8, 10, 12, 14\}$ é efetuado um total de 17 consultas a elementos da árvore.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: A árvore é perfeitamente equilibrada, com altura 2. Qualquer pesquisa (existente ou não) percorre no máximo 3 nós. Somando as consultas para todos os elementos indicados obtém-se 17 acessos.

Sugiro que desenhe neste espaço a árvore.

15. Considere um grafo orientado $G(V, E)$ representado usando a lista ordenada dos seus nós e, para cada nó, a sua lista ordenada de adjacências:

- (a) No pior caso, verificar se um nó é isolado é uma op. de complexidade $O(E)$.
- (b) No pior caso, adicionar uma aresta é uma operação de complexidade $O(V)$.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: Para verificar se um nó é isolado pode ser necessário percorrer todas as listas. Adicionar uma aresta pode exigir inserir ordenadamente na lista de adjacências, o que no pior caso é $O(V)$.

16. A teoria da complexidade computacional classifica diferentes problemas em classes de complexidade:

- (a) **A classe P contém todos os problemas de decisão que podem ser resolvidos, em tempo polinomial, por um algoritmo determinista.**
- (b) A classe NP contém todos os problemas de decisão cuja solução não pode ser verificada, em tempo polinomial, por um algoritmo determinista.
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.

Resolução: NP contém problemas cujas soluções **podem** ser verificadas em tempo polinomial. Logo, apenas a primeira afirmação é correta.

17. Em C++, a *keyword* `auto` permite:

- (a) **Deduzir, de modo automático, o tipo de uma variável em tempo de compilação.**
- (b) Inicializar, de modo automático, uma variável com o seu *default* value.
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.

Resolução: `auto` apenas permite dedução de tipo; não inicializa valores.

18. Qual das seguintes estruturas de dados (C++ *containers*) fornece operações eficientes — isto é, com complexidade $O(1)$ — tanto para a inserção como para a remoção de elementos nas duas extremidades?

- (a) `std::vector`.
- (b) **`std::deque`.**
- (c) `std::queue`.
- (d) Nenhuma está correta.

Resolução: O `std::deque` foi concebido para operações eficientes em ambas as extremidades... `std::vector` é $O(1)$ só no fim.

19. Qual é a característica principal da estrutura de dados (C++ *container*) `std::set`?

- (a) Permite armazenar elementos duplicados.
- (b) Permite o acesso aleatório aos elementos armazenados.
- (c) **Mantém os seus elementos ordenados.**
- (d) Nenhuma está correta.

Resolução: Em C++, o `std::set` é implementado como uma árvore balanceada, pelo que garante ordem crescente e unicidade...

20. Em C++, o algoritmo `std::count_if` permite:

- (a) Contar o número de ocorrências de um elemento específico (*target*) de uma estrutura de dados (*container*).
- (b) **Contar quantos elementos de uma estrutura de dados (*container*) satisfazem uma dada condição.**
- (c) Contar os elementos únicos de uma estrutura de dados (*container*).
- (d) Nenhuma está correta.

Resolução: `std::count_if` aplica um predicado a cada elemento e conta os que o satisfazem.

Exame EN de 2024/2025

1. Sejam dadas duas funções $t(n)$ e $g(n)$, uma constante positiva c e um inteiro não negativo n_0 :

- (a) se $t(n) < c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$, i.e., $t(n)$ pertence a $O(g(n))$.
- (b) se $t(n) < c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $\Omega(g(n))$, i.e., $t(n)$ pertence a $\Omega(g(n))$.
- (c) se $t(n) \geq c g(n)$, para todo o $n > n_0$, então $t(n)$ é da ordem de $O(g(n))$, i.e., $t(n)$ pertence a $O(g(n))$.
- (d) **Nenhuma está correta.**

Resolução: A definição correta de $O(g(n))$ exige $t(n) \leq c g(n)$. Nenhuma das afirmações apresenta corretamente as definições de O ou Ω .

2. Considere muitas instâncias distintas de um mesmo problema, todas da mesma dimensão e que foram resolvidas usando o mesmo algoritmo.

- (a) Se o esforço computacional para resolver cada instância **não** foi idêntico, fará sentido analisar a eficiência computacional do algoritmo para o pior caso e o caso médio.
- (b) Se o esforço computacional para resolver cada uma dessas instâncias **não** dependeu da configuração de cada instância, não faz qualquer sentido falar em pior caso e em caso médio.
- (c) **Ambas estão corretas.**
- (d) Nenhuma está correta.

Resolução: Se o esforço varia entre instâncias, faz sentido falar em pior caso e caso médio. Se não depende da configuração, esses conceitos não se aplicam.

3. A tabela seguinte representa o número de operações básicas efetuadas para sucessivos valores de n :

n	1	2	4	8	16	32	64	128	256	512	1024
$M(n)$	0	1	3	5	7	9	11	13	15	17	19

Trata-se de um algoritmo com ordem de complexidade:

- (a) **logarítmica.**
- (b) linear.
- (c) quadrática.
- (d) Nenhuma está correta.

Resolução: Sempre que n duplica, $M(n)$ cresce de forma linear, caracterizando $O(\log n)$.

4. A tabela seguinte representa o número de operações básicas efetuadas para sucessivos valores de n :

n	1	2	4	8	16	32	64	128	256
$M(n)$	3	7	21	73	273	1057	4161	16513	65793

Trata-se de um algoritmo com ordem de complexidade:

- (a) logarítmica.
- (b) linear.
- (c) **quadrática.**
- (d) Nenhuma está correta.

Resolução: Os valores crescem aproximadamente com n^2 , logo a complexidade é quadrática.

5. Quando se usa um algoritmo de procura exaustiva (*exhaustive search*), é geralmente necessário enumerar todas as possíveis soluções de um conjunto de n elementos; um passo essencial para a resolução do problema envolve a geração de todas as permutações desses elementos. Um exemplo de um problema onde isto é aplicado é (**Nota: enunciado estava muito desfocado, portanto não é exatamente assim, mas consegui +/- decifrar a questão**):

- (a) Problema da Mochila (*The Knapsack Problem*).
- (b) **Problema do Caixeiro Viajante (*The Traveling Salesman Problem*).**
- (c) Ambas estão corretas.
- (d) Nenhuma está correta.

Resolução: O TSP requer a geração de todas as permutações possíveis. O Problema da Mochila envolve a geração de subconjuntos (combinações), e não a geração de todas as permutações dos n elementos.

6. Em C++, o algoritmo `std::count` permite:

- (a) **Contar o número de ocorrências de um elemento específico (*target*) de uma estrutura de dados (*container*).**
- (b) Contar quantos elementos de uma estrutura de dados (*container*) satisfazem uma dada condição.
- (c) Contar os elementos únicos de uma estrutura de dados (*container*).
- (d) Nenhuma está correta.

Resolução: `std::count` compara diretamente com um valor alvo.

Verdadeiro / Falso

1. Um algoritmo é definido por uma sequência de instruções possivelmente ambíguas, que permite resolver instâncias de um problema num espaço de tempo finito.

Falso — Algoritmos devem ser não ambíguos.

2. Uma função $t(n)$ pertence a $\Theta(g(n))$ se existirem constantes $c_1, c_2 > 0$ e n_0 tal que $c_1 g(n) \leq t(n) \leq c_2 g(n)$, para todo o $n > n_0$.

Verdadeiro — Esta é a definição formal de $\Theta(g(n))$.

3. Ao efetuar, usando o algoritmo clássico, o produto de matrizes $A \times B = C$, em que a matriz A tem 10 linhas e 20 colunas e a matriz B tem 20 linhas e 10 colunas, são efetuadas **2000** multiplicações.

Verdadeiro — $10 \times 20 \times 10 = 2000$ multiplicações.

4. Um algoritmo que gera todas as permutações de um conjunto de n elementos é um algoritmo de ordem de complexidade exponencial, i.e., pertence a $O(2^n)$.

Falso — Permutações têm complexidade $O(n!)$, não $O(2^n)$.

5. No caso médio, a versão **iterativa** do algoritmo de Pesquisa Binária num *array* ordenado tem complexidade $O(\log n)$.

Verdadeiro — Pesquisa binária é $O(\log n)$.

6. Quando se usa a técnica da programação dinâmica, é necessário resolver repetidas vezes os mesmos sub-problemas.

Falso — Programação dinâmica evita resolver subproblemas repetidos.

7. $0 + 3 + 6 + \dots + 3(n-1) + 3n = 3(n^2 - n)/n$.

Falso — $3(n^2 - n)/2$.

8. $1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1$.

Verdadeiro — Soma geométrica.

9. Quando se ordena um dado *array* de n elementos usando o algoritmo *HeapSort*, o primeiro passo é, habitualmente, transformar o *array* dado numa **MIN-HEAP**.

Falso — *HeapSort* começa com **MAX-HEAP**.

10. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de **modo aleatório**. Se for usado o algoritmo *HeapSort*, a ordem de complexidade dessa tarefa será $O(n \log n)$.

Verdadeiro — *HeapSort* é $O(n \log n)$.

11. Numa árvore binária, o número máximo de nós do nível i é 2^i , considerando que a raiz da árvore pertence ao nível zero ($i=0$).

Verdadeiro — Máximo 2^i nós no nível i .

12. Na travessia **pós-ordem** de uma árvore binária, todos os elementos da subárvore direita da raiz são visitados antes dos elementos da subárvore esquerda da raiz.

Falso — Pós-ordem visita esquerda antes da direita.

13. Numa árvore binária de pesquisa ("Binary Search Tree"), a subárvore esquerda de um dado nó **não** pode conter elemento de valor superior a esse mesmo nó.
Verdadeiro — Isso é uma propriedade fundamental da BST.
14. Uma árvore AVL é uma árvore binária equilibrada em altura em que, para cada nó, as alturas das suas duas subárvores diferem, **sempre**, de uma unidade.
Falso — AVL admite diferença **até** 1, não sempre 1.
15. Um grafo (não orientado) completo com n vértices tem $(n^2 - n)/2$ arestas.
Verdadeiro — Fórmula correta...
16. Se um grafo orientado é **fortemente conexo**, não tem qualquer vértice isolado.
Verdadeiro — Forte conexidade implica ausência de isolados.
17. Dado um grafo, um circuito Hamiltoniano é um caminho que, partindo de um qualquer nó, atravessa, uma única vez, cada um dos outros vértices do grafo e regressa ao inicial.
Verdadeiro — Definição de circuito Hamiltoniano...
18. Considere um grafo orientado $G(V, E)$ representado usando uma lista de vértices e, para cada vértice, a sua lista de adjacências. Se o grafo for completo, existem V listas de adjacências e o numero total de nós definindo as várias listas de adjacências é $V \times (V - 1)$.
Verdadeiro — $V(V - 1)$ adjacências num grafo orientado completo.
19. A travessia por níveis ("Breadth-First Traversal") de um grafo orientado é habitualmente realizada usando uma pilha ("Stack").
Falso — BFS usa fila ("Queue"), não pilha.
20. Instâncias de grande dimensão do Problema do Caixeiro Viajante ("The Traveling Salesperson Problem") **não** são habitualmente resolúveis em tempo útil.
Verdadeiro — TSP é intratável para grandes instâncias (por procura exaustiva)...

Exame EN de 2024/2025

1. Um algoritmo é definido por uma sequência de instruções **não-ambíguas** que permite resolver instâncias de um problema num espaço de tempo **indefinido**.

Falso – Um algoritmo é definido como uma sequência de instruções **não-ambíguas** que resolve instâncias de um problema num **espaço de tempo finito** $\neq$ indefinido.

2. Uma função $t(n)$ será da ordem $\Omega(g(n))$, i.e. $t(n)$ pertence a $\Omega(g(n))$, se existir uma constante positiva c e um inteiro não-negativo n_0 tais que $cg(n) \leq t(n)$, para todo $n > n_0$.

Verdadeiro – Por definição, $t(n) \in \Omega(g(n))$ se existir uma constante positiva c e um inteiro n_0 tais que $t(n) \geq cg(n)$, $\forall n > n_0$.

3. Seja $t(n) = 1000n^2 + n^3$. Então, $t(n)$ é da ordem de $O(n^3)$, i.e., $t(n)$ pertence a $O(n^3)$.

Verdadeiro – O termo dominante é n^3 . Logo, $t(n) \in O(n^3)$.

4. Ao efetuar, usando o algoritmo clássico, o produto de matrizes $(A \times B) \times C = D$, em que a matriz A tem 2 linhas e 3 colunas, a matriz B tem 3 linhas e 4 colunas e a matriz C tem 4 linhas e 5 colunas, são efetuadas, **no total, 120** multiplicações.

Falso – O produto $A \times B$ envolve: $2 \times 3 \times 4 = 24$ multiplicações. O resultado tem dimensão 2×4 . Multiplicando depois por C : $2 \times 4 \times 5 = 40$ multiplicações. No total: $24 + 40 = 64 \neq 120$.

5. Pretende-se ordenar um dado *array* de n elementos, todos distintos, e que se encontram armazenados de **modo aleatório**. Se for usado o algoritmo **Selection Sort**, e consideradas o número de comparações efetuadas, a ordem de complexidade dessa tarefa será $O(n^2)$.

Verdadeiro – $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$, o que corresponde a uma complexidade $O(n^2)$.

6. Um algoritmo que gera todos os **subconjuntos** de um conjunto de n elementos é um algoritmo de ordem de complexidade exponencial, i.e., pertence a $O(2^n)$.

Verdadeiro – O número de subconjuntos de um conjunto com n elementos é 2^n .

7. A técnica de Programação Dinâmica pode ser utilizada para desenvolver um algoritmo para determinar as combinações $C(n, p)$ – i.e., os coeficientes binomiais.

Verdadeiro – Os coeficientes binomiais $C(n, p)$ satisfazem a recorrência: $C(n, p) = C(n-1, p-1) + C(n-1, p)$, e são uma aplicação bem-conhecida da DP.

8. A combinação das técnicas de Programação Dinâmica e "memoization" é útil, mas continuam a ser executados cálculos **redundantes**.

Falso – Quando a DP é combinada com *memoization*, os resultados de subproblemas são armazenados e reutilizados.

9. $0 + 10 + 20 + 30 + \dots + 10(n-1) + 10n = 5(n^2 + n)$.

Verdadeiro – $10 \times n(n+1)/2 = 5(n^2 + n)$.

10. $0 + 1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1$.

Verdadeiro – P.G....

Parte B

Propostas de Resolução

Nota: Estas são apenas **sugestões** de resolução. Existe mais do que 1 forma de apresentar um algoritmo correto para cada uma das questões... Para verificarem se a vossa resolução (em papel) está correta, poderão, pex, transcrevê-la para uma IDE e desenvolverem um pequeno programa de teste (com, pex, o exemplo apresentado no enunciado) e verem se compila e, compilando, se oferece a funcionalidade desejada.

Se detetar algum erro, avise-me, por favor, para eu corrigir!

Exame AED 2023-2024 EN

Q1

a)

```
void SelectionSort( int* a, unsigned int n ) {
    for (unsigned int i = 0; i < n - 1; i++) {
        unsigned int minIndex = i;
        for(unsigned int j = i +1; j < n; j++)
            if (a[j] < a[minIndex]) minIndex = j;
        if (minIndex != i) swap( &a[i], &a[minIndex] );
    }
}

static void swap(int* e1, int* e2) {
    int temp = *e1;
    *e1 = *e2;
    *e2 = temp;
}
```

b)

$$\sum_{i=0}^{N-1} \sum_{j=i+1}^N 1 = \sum_{i=0}^{N-1} N - i = N(N+1)/2 \rightarrow O(N^2)$$

c)

Melhor caso, 0 trocas, $O(1) \rightarrow$ *array* já ordenado.

Pior caso, $n-1$ trocas, $O(n) \rightarrow$ *array* invertido.

Caso médio, $O(n)$ também (cerca de $n/2$ trocas) $\rightarrow$ pex, [1, 3, 2, 4, 6, 5]

Q2

a)

```
header_pointer Merge( header_pointer seq_1, header_pointer seq_2 ) {
    Node *node1 = seq_1 ? seq_1->first : NULL;
    Node *node2 = seq_2 ? seq_2->first : NULL;
    header_pointer mergeList = malloc(sizeof(*mergeList));
    mergeList->first = NULL; mergeList->numElems = 0;

    Node *nodeMerged = NULL;
    while (node1 && node2) {
        Node *newNode = malloc(sizeof(*newNode)); newNode->next = NULL;

        if (node1->value <= node2->value) {
            newNode->value = node1->value; node1 = node1->next; }
        else { newNode->value = node2->value; node2 = node2->next; }

        if (!mergeList->first) mergeList->first = newNode;
        else nodeMerged->next = newNode;

        nodeMerged = newNode; mergeList->numElems++;
    }

    while (node1) {
        Node *newNode = malloc(sizeof(*newNode)); newNode->next = NULL;
        newNode->value = node1->value;

        if (mergeList->first == NULL) mergeList->first = newNode;
        else nodeMerged->next = newNode;

        nodeMerged = newNode; mergeList->numElems++; node1 = node1->next;
    }
    while (node2) {
        Node *newNode = malloc(sizeof(*newNode)); newNode->next = NULL;
        newNode->value = node2->value;

        if (mergeList->first == NULL) mergeList->first = newNode;
        else nodeMerged->next = newNode;

        nodeMerged = newNode; mergeList->numElems++; node2 = node2->next;
    }

    return mergeList;
}
```

b)

Pior caso: $O(n_1 + n_2)$.

Melhor caso: $O(\min(n_1, n_2))$.

Q3

a)

```
unsigned int CountNonLeafNodes( pointer root ) {  
    if (root == NULL) return 0;  
    if (root->leftSubTree == NULL && root->rightSubTree == NULL) return 0;  
  
    return 1 +  
        CountLeafNodes(root->leftSubTree) +  
        CountLeafNodes(root->rightSubTree);  
}
```

b)

Travessia pre-order (pré-orden).

c)

```
unsigned int CountValuesInRange( pointer root, int a, int b ) {  
    if (root == NULL) return 0;  
    if (root->value < a) return CountValuesInRange(root->rightSubTree, a, b);  
    if (root->value > b) return CountValuesInRange(root->leftSubTree, a, b);  
  
    return 1 +  
        CountValuesInRange(root->leftSubTree, a, b) +  
        CountValuesInRange(root->rightSubTree, a, b);  
}
```

Exame AED 2024-2025 EN

Q1

a)

```
static inline int isEven(unsigned int num) { return !(num & 1); }

unsigned int* Partition(unsigned int* a, unsigned int n) {
    unsigned int* result = (unsigned int*) malloc(n * sizeof(unsigned int));

    int evenCount = 0, oddCount = 0;
    for( int i = 0; i < n; i++ ) {
        if ( isEven(a[i]) ) evenCount++;
        else oddCount++;
    }
    unsigned int* even = (unsigned int*)malloc(evenCount * sizeof(unsigned int));
    unsigned int* odd = (unsigned int*)malloc(oddCount * sizeof(unsigned int));

    int evenIndex = 0, oddIndex = 0;
    for( int i = 0; i < n; i++ ) {
        if ( isEven(a[i]) ) even[evenIndex++] = a[i];
        else odd[oddIndex++] = a[i];
    }

    for( int i = 0; i < oddCount; i++ ) result[i] = odd[i];

    int evenIn = n - 1;
    for( int i = 0; i < evenCount; i++ ) result[evenIn--] = even[i];

    free(even); free(odd);
    return result;
}
```

Q2

a)

```
unsigned int DeleteRepetitions(pointer p) {
    unsigned int deletedCounter = 0;

    pointer curNode = p;
    while(curNode) {
        pointer prevNode = curNode;
        pointer analyzeNode = curNode->next;

        while(analyzeNode) {
            if(analyzeNode->value == curNode->value) {
                prevNode->next = analyzeNode->next;
                free(analyzeNode);
                analyzeNode = prevNode->next;
                deletedCounter++;
            } else {
                prevNode = analyzeNode;
                analyzeNode = analyzeNode->next;
            }
        }

        curNode = curNode->next;
    }

    return deletedCounter;
}
```

b)

No melhor caso, todos os elementos têm valores diferentes (não há valores repetidos), portanto não há trocas. No entanto, este algoritmo é sistemático no número de visitas (sempre o mesmo número de visitas, independentemente da configuração). $\sum_{i=0}^{i=N=10} 1 = 55$ visitas.

c)

O pior caso é os elementos serem todos iguais, logo há 9 trocas, mas o número de visitas é o mesmo.

Q3

Falta-me esta questão para este exame (EN 2024-2025). Não consegui encontrá-la... Caso consiga, por favor avise-me.

Exame AED 2024-2025 ER

Q1

Falta-me esta questão para este exame (ER 2024-2025). Não consegui encontrá-la... Caso consiga, por favor avise-me.

Q2

```
pointer Partition(pointer p) {
    if (!p) return NULL;

    pointer evenHead = NULL, evenTail = NULL;
    pointer oddHead = NULL, oddTail = NULL;

    pointer current = p;
    while (current) {
        pointer novo = (pointer)malloc(sizeof(struct node));
        novo->value = current->value;
        novo->next = NULL;

        if (current->value % 2 == 0) {
            novo->next = evenHead;
            evenHead = novo;

            if (!evenTail) evenTail = novo;
        } else {
            if (!oddHead) {
                oddHead = novo;
                oddTail = novo;
            } else {
                oddTail->next = novo;
                oddTail = novo;
            }
        }

        current = current->next;
    }

    if (!evenHead) return oddHead;
    if (!oddHead) return evenHead;

    evenTail->next = oddHead;
    return evenHead;
}
```

Q3

a)

```
unsigned int Search(pointer root, unsigned int number) {
    while (root) {
        if (root->value == number) return 1;
        else if (number < root->value) root = root->left;
        else root = root->right;
    }
    return 0;
}
```

b)

```
unsigned int CountContainedIn(pointer root1, pointer root2) {
    if (!root1) return 0;

    return Search(root2, root1->value)
        + CountContainedIn(root1->left, root2)
        + CountContainedIn(root1->right, root2);
}
```

Exercícios-Tipo feitos por mim

Vetores

Q1

a)

```
static inline int isFiveMultiple(unsigned int num) { return num % 5 == 0; }
static inline int isOdd(unsigned int num) { return num & 1; }

unsigned int* PartitionThree(unsigned int* a, unsigned int n) {
    unsigned int* result = (unsigned int*) malloc(n * sizeof(unsigned int));

    int numMod5 = 0, numEvenNotMod5 = 0, numOddNotMod5 = 0;
    for(int i = 0; i < n; i++) {
        if( isFiveMultiple(a[i] )) numMod5++;
        else if ( isOdd( a[i] )) numOddNotMod5++;
        else numEvenNotMod5++;
    }

    unsigned int* mod5 = (unsigned int*)malloc(numMod5 * sizeof(unsigned int));
    unsigned int* evenNotMod5 = (unsigned int*)malloc(numEvenNotMod5 * sizeof(unsigned int));
    unsigned int* oddNotMod5 = (unsigned int*)malloc(numOddNotMod5 * sizeof(unsigned int));

    int indexMod5 = 0, indexEvenNotMod5 = 0, indexOddNotMod5 = 0;
    for(int i = 0; i < n; i++) {
        if( isFiveMultiple(a[i] )) mod5[indexMod5++] = a[i];
        else if ( isOdd( a[i] )) oddNotMod5[indexOddNotMod5++] = a[i];
        else evenNotMod5[indexEvenNotMod5++] = a[i];
    }

    int resultIndex = 0;
    for(int i = 0; i < numMod5; i++)
        result[resultIndex++] = mod5[i];

    for(int i = numEvenNotMod5 - 1; i >= 0; i--)
        result[resultIndex++] = evenNotMod5[i];

    for(int i = numOddNotMod5 - 1; i >= 0; i--)
        result[resultIndex++] = oddNotMod5[i];

    free(mod5); free(evenNotMod5); free(oddNotMod5);
    return result;
}
```

b) $4n$ comparações ($\text{isFiveMultiple}(a[i])$ e $\text{isOdd}(a[i])$), $2n$ na primeira passagem para determinar o num. de elems de cada partição e $+2n$ na segunda passagem para atribuir os nums. respetivos a cada subarray. não há "casos" - o número de comps é sistemático.

c) O num. de escritas também é sistemático: n escritas.

Q2

a)

```
static inline int isPrime(unsigned int n) {
    if (n < 2) return 0;
    for (unsigned int i = 2; i*i <= n; i++) if (n % i == 0) return 0;
    return 1;
}

unsigned int* PrimeStablePartition(unsigned int* a, unsigned int n) {
    unsigned int* result = (unsigned int*) malloc(n * sizeof(unsigned int));
    unsigned int k = 0;

    for (unsigned int i = 0; i < n; i++)
        if (isPrime(a[i])) result[k++] = a[i];

    unsigned int* nonPrimes = (unsigned int*) malloc(n * sizeof(unsigned int));
    unsigned int m = 0;
    for (unsigned int i = 0; i < n; i++)
        if (!isPrime(a[i])) nonPrimes[m++] = a[i];

    for (unsigned int i = 1; i < m; i++) {
        unsigned int tmp = nonPrimes[i];
        int j = i - 1;
        while (j >= 0 && nonPrimes[j] > tmp) {
            nonPrimes[j + 1] = nonPrimes[j]; j--;
        }
        nonPrimes[j + 1] = tmp;
    }

    for (unsigned int i = 0; i < m; i++) result[k++] = nonPrimes[i];

    free(nonPrimes); return result;
}
```

b)

Para verificar "primalidade" são feitas $\sum_{i=0}^{n-1} O(\sqrt{a[i]})$ comparações.

Para a ordenação dos não primos, são feitas, com m não primos: $\sum_{i=1}^{m-1} i = \frac{m(m-1)}{2}$

O total de comparações é: $O\left(\sum_{i=0}^{n-1} \sqrt{a[i]} + m^2\right)$

A ordem de complexidade temporal (em termos de n) é:

$O(n \cdot \sqrt{\max(a[i])} + m^2) \subseteq O(n^2)$ no pior caso se $m = O(n)$

c)

No **Melhor caso** (vetor já primos no início.), todos os elementos são primos, não há escritas adicionais para ordenar, logo o número de escritas é n . No **Pior caso** (só não-primos em ordem decrescente), todos os elementos são não-primos, necessitam de ser ordenados; o número de escritas é $\approx n + m^2$ (insertion sort) $= O(n^2)$. No **Caso médio**: metade primos e metade não primos; número de escritas $= n/2$ (copiar primos) $+ m^2/2$ (ordenar não primos) $\approx O(n^2/4)$.

Q3

a)

```
unsigned int* InterleavedMerge(unsigned int* v1, unsigned int n1,
                               unsigned int* v2, unsigned int n2) {
    unsigned int* result = (unsigned int*) malloc((n1 + n2) * sizeof(unsigned int));
    unsigned int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2) {
        if (v1[i] <= v2[j]) { // empate
            result[k++] = v1[i++];
        } else {
            result[k++] = v2[j++];
        }
    }

    while (i < n1) result[k++] = v1[i++];
    while (j < n2) result[k++] = v2[j++];

    return result;
}
```

b)

Comparações principais: a cada passo da fusão (while $i < n_1$ e $j < n_2$) fazemos $v1[i] \leq v2[j]$. Número total de comparações:

- **Melhor Caso:** todos os elementos de um vetor são menores que os do outro. Cada comparação seleciona sempre um elemento de um vetor até esgotá-lo. Número de comparações: $\min(n_1, n_2)$.
- **Pior Caso:** os elementos alternam-se ou existem muitos empates, de forma que cada comparação escolhe apenas um elemento, e a outra lista ainda tem elementos. Número de comparações: $n_1 + n_2 - 1$.

Ordem de complexidade: $O(n_1 + n_2)$ em todos os casos.

Q4

a)

```
unsigned int CompressArray(unsigned int* a, unsigned int n) {
    if (n == 0) return 0;

    unsigned int writeIndex = 1;

    for (unsigned int readIndex = 1; readIndex < n; readIndex++) {
        if (a[readIndex] != a[readIndex - 1]) {
            a[writeIndex] = a[readIndex];
            writeIndex++;
        }
    }

    return writeIndex;
}
```

b)

A cada iteração do `for`, é efetuada exatamente uma comparação: `a[readIndex] != a[readIndex - 1]`, pelo que o número total de comparações é $N - 1$ e a ordem de complexidade temporal $O(N)$.

c)

Cada vez que encontramos um novo elemento distinto, fazemos uma escrita no vetor: `a[writeIndex] = a[readIndex]`.

Melhor Caso: todos os elementos são iguais. Apenas uma posição é escrita (o primeiro elemento) e não há outras escritas. Número de escritas: 1.

Pior Caso: nenhum elemento repetido. Cada elemento é escrito apenas uma vez no local correto (exceto o primeiro que já está na posição correta). Número de escritas: $N - 1$.

Ordem de complexidade das escritas: $O(1)$ no Melhor Caso, $O(N)$ no Pior Caso.

Exercícios-Tipo feitos por mim

Listas Ligadas

Q1

a)

```
pointer PartitionList(pointer p) {
    pointer negHead = NULL, negTail = NULL;
    pointer posHead = NULL, posTail = NULL;
    pointer cur = p;

    while (cur) {
        pointer nextNode = cur->next;
        cur->next = NULL;

        if(cur->value < 0) {
            if (!negHead) { negHead = cur = negTail; }
            else { negTail->next = cur; negTail = cur; }
        } else {
            if (!posHead) { posHead = posTail = cur; }
            else { posTail->next = cur; posTail = cur; }
        }

        cur = nextNode;
    }

    if (negTail) negTail->next = posHead;
    return negHead ? negHead : posHead;
}
```

b)

$3n$ vezes (1) $cur \neq NULL$ no loop principal, 2) $cur->value < 0$ e 3) $!neg/!posHead$), e é sistemático (não há casos diferentes).

c)

Há sempre religação exceto para o primeiro elemento par/ímpar (em que a cabeça e cauda ainda não têm valores definidos). Assim, o número de religações será $2n$ (início do loop) + $n - 2$ (primeiro elemento par/ímpar). No caso de serem alternados, o "pior caso", será acrescido de 1, pela religação final.

Q2

a)

```
unsigned int RemoveEvenPositions(pointer head) {
    pointer prev = head;
    pointer cur = head->next;
    unsigned int count = 0;
    unsigned int pos = 2; // pos os present node

    while (cur) {
        // deletes even pos node
        prev->next = cur->next;
        pointer temp = cur;
        cur = cur->next;
        free(temp);
        count++;

        // next node
        if (cur) {
            prev = cur;
            cur = cur->next;
            pos += 2;
        }
    }

    return count;
}
```

b)

Cada nó da lista é visitado exatamente uma vez para decidir se será removido ou não, pelo que o número total de visitas a nós é N , independentemente da configuração da lista. O melhor caso é qualquer lista, pois o número de visitas é sempre o mesmo. Pior Caso, idem.

Ordem de complexidade temporal é $O(N)$ e Ordem de complexidade espacial é $O(1)$, pois apenas alguns ponteiros auxiliares são usados.

Q3

a)

```
pointer ThreeWayStablePartition(pointer head) {
    pointer negHead = NULL, negTail = NULL;
    pointer zeroHead = NULL, zeroTail = NULL;
    pointer posHead = NULL;

    pointer cur = head;
    while (cur) {
        pointer nextNode = cur->next;
        cur->next = NULL;

        if (cur->value < 0) {
            if (!negHead) negHead = negTail = cur;
            else { negTail->next = cur; negTail = cur; }
        } else if (cur->value == 0) {
            if (!zeroHead) zeroHead = zeroTail = cur;
            else { zeroTail->next = cur; zeroTail = cur; }
        } else {
            cur->next = posHead;
            posHead = cur;
        }
        cur = nextNode;
    }

    pointer newHead = NULL, tail = NULL;

    if (negHead) { newHead = negHead; tail = negTail; }
    if (zeroHead) {
        if (!newHead) newHead = zeroHead;
        else tail->next = zeroHead;
        tail = zeroTail;
    }
    if (posHead) {
        if (!newHead) newHead = posHead;
        else tail->next = posHead;
    }

    return newHead;
}
```

b)

Cada nó é visitado exatamente uma vez, para se determinar se é negativo, zero ou positivo, pelo que o número total de comparações é N e a ordem de complexidade $O(N)$.

c)

Cada nó tem o seu **next** atualizado exatamente uma vez. Após processar todos os nós, são necessárias até 2 conexões para juntar as sublistas.

Melhor e Pior Caso: $N + 2$ religações de ponteiros e ordem de complexidade $O(N)$

Q4

a)

```
pointer ReverseInBlocks(pointer head, unsigned int K) {
    if (!head || K == 1) return head;

    pointer dummy = malloc(sizeof(struct node));
    dummy->next = head;
    pointer prevBlock = dummy;
    pointer curr = head;

    while (curr) {
        pointer check = curr;
        for (int i = 0; i < K; i++) {
            if (!check) return dummy->next;
            check = check->next;
        }

        pointer prev = NULL;
        pointer blockStart = curr;
        for (int i = 0; i < K; i++) {
            pointer nextNode = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nextNode;
        }

        prevBlock->next = prev;
        blockStart->next = curr;
        prevBlock = blockStart;
    }

    pointer newHead = dummy->next;
    free(dummy);
    return newHead;
}
```

b)

Cada nó da lista é visitado exatamente **uma vez** para inversão, independentemente do bloco. Se a lista tem N nós, o número total de visitas é N e a ordem de complexidade $O(N)$.

c)

Cada nó dentro de um bloco tem seu ponteiro **next** alterado **uma vez** durante a inversão. Existem N nós, então o número total de atribuições de ponteiros é também N ...

Cada bloco requer também duas conexões externas (início e fim do bloco) com os nós adjacentes, mas como há N/K blocos, isso adiciona $2 \cdot (N/K)$ atribuições.

Portanto, o número total de atribuições de ponteiros é $N + 2 \cdot \frac{N}{K} = N \left(1 + \frac{2}{K}\right)$.

Exercícios-Tipo feitos por mim

Árvores Binárias

Q1

a)

```
int SumLeafValues(pointer root) {  
    if (root == NULL) return 0;  
  
    if (root->left == NULL && root->right == NULL) {  
        return root->value;  
    }  
  
    return SumLeafValues(root->left) + SumLeafValues(root->right);  
}
```

b)

A função realiza uma travessia **pré-ordem implícita**, no sentido de visitar o nó atual e depois recorrer recursivamente às subárvores esquerda e direita. No entanto, como a soma é apenas das folhas, a visita ao nó só contribui se for folha, mas a ordem de chamada recursiva é *left-right*.

c)

- **Melhor caso:** A árvore é altamente desequilibrada (uma única folha à esquerda ou direita) e a função visita apenas os nós necessários. **Complexidade:** $O(h)$, onde h é a altura da árvore (mínima quantidade de nós até a folha relevante).
- **Pior caso:** A árvore é completamente balanceada e a função precisa visitar todos os nós para identificar quais são folhas. **Complexidade:** $O(n)$, sendo n o número total de nós da árvore.

Q2

a)

```
unsigned int CountLessThan(pointer root, int X) {
    if (root == NULL) return 0;

    if (root->value >= X) {
        return CountLessThan(root->left, X);
    } else {
        return 1 +
            CountLessThan(root->left, X) +
            CountLessThan(root->right, X);
    }
}
```

b)

É possível evitar explorar subárvores inteiras porque uma BST garante que:

Todos os elementos da subárvore esquerda de um nó $\leq$ nó,

Todos os elementos da subárvore direita de um nó $\geq$ nó.

Portanto:

- Se o nó atual $\geq X$, não precisamos de visitar a subárvore direita.
- Se o nó atual $< X$, sabemos que todos os elementos da subárvore esquerda podem ser considerados menores e podemos explorar a direita para continuar a contagem.

c)

- **Melhor caso:** A BST está **balanceada** e X é maior que todos os elementos. **Número mínimo de nós visitados:** $O(\log n)$ por caminho da raiz até a folha relevante.
- **Pior caso:** A BST é completamente **degenerada** (como uma lista ligada) e X exige visitar todos os nós. **Número máximo de nós visitados:** $O(n)$.

Q3

a)

```
pointer BuildBSTFromPreOrderHelper(int* preOrder, int start, int end) {
    if (start > end) return NULL;

    pointer root = (pointer)malloc(sizeof(struct node));
    root->value = preOrder[start];
    root->left = root->right = NULL;

    int split = start + 1;
    while (split <= end && preOrder[split] < root->value) split++;

    root->left = BuildBSTFromPreOrderHelper(preOrder, start + 1, split - 1);
    root->right = BuildBSTFromPreOrderHelper(preOrder, split, end);

    return root;
}

pointer BuildBSTFromPreOrder(int* preOrderArray, unsigned int n) {
    return BuildBSTFromPreOrderHelper(preOrderArray, 0, n - 1);
}
```

b)

- **Configuração da ABP:** O melhor caso ocorre quando a ABP está **balanceada**, ou seja, cada nó possui aproximadamente o mesmo número de nós nas subárvores esquerda e direita.
- **Complexidade:** No melhor caso, cada divisão do vetor ocorre aproximadamente ao meio. Para n elementos:

$$T(n) = 2T(n/2) + O(1) \implies T(n) = O(n)$$

porque cada elemento é processado exatamente uma vez para criar o nó e determinar o ponto de divisão.

Nota: Se a árvore fosse degenerada (uma lista), o número de comparações seria $O(n^2)$.

c)

- Para uma árvore binária genérica, a travessia pré-ordem sozinha não é suficiente, pois não há garantias sobre a ordem relativa dos elementos.
- É necessário conhecer **mais uma travessia**, por exemplo:
 - Pré-ordem + In-ordem
 - Pós-ordem + In-ordem
- A travessia adicional (in-order) permite determinar a posição relativa dos elementos à esquerda e à direita da raiz, permitindo reconstruir unicamente a árvore.

Q4

a)

```
int MaxDepth(pointer root) {
    if (!root) return 0;
    int leftDepth = MaxDepth(root->left);
    int rightDepth = MaxDepth(root->right);
    return (leftDepth > rightDepth ? leftDepth : rightDepth) + 1;
}

int SumLevel(pointer root, int level) {
    if (!root) return 0;
    if (level == 1) return root->value;
    return SumLevel(root->left, level-1) + SumLevel(root->right, level-1);
}

int SumMaxDepth(pointer root) {
    int depth = MaxDepth(root);
    return SumLevel(root, depth);
}
```

b)

- **Tempo:** $O(N)$, pois cada nó é visitado exatamente uma vez.
- **Espaço:** $O(h)$, com h altura da árvore (pilha de chamadas recursivas).

c)

Basta somar os filhos da raiz diretamente:

```
int SumLevel1(pointer root) {
    if (!root) return 0;
    int sum = 0;
    if (root->left) sum += root->left->value;
    if (root->right) sum += root->right->value;
    return sum;
}
```

Complexidade: $O(1)$ tempo e $O(1)$ espaço, pois acessamos só os filhos diretos da raiz.